RISC – V Single Cycle Processor

Name: Gautam Raj Chhabra
Project: 32-bit RISC-V Processor
Language: Verilog HDL, SystemVerilog
Tools Used: Cadence Xcelium, Vivado Xilinx, EDA Playground

1. Abstract

This project implements a 32-bit RISC-V single-cycle processor using Verilog HDL.
The processor consists of Program Counter, Instruction Memory, Register File, ALU, Immediate Generator, Control Unit, Data Memory, and Multiplexers. Each instruction is executed in one clock cycle. The design is verified using SystemVerilog testbench and simulated using EDA Playground, Vivado, and Cadence Xcelium. Waveforms, RTL schematic, and synthesis results confirm correct functionality of the processor.

2. Introduction

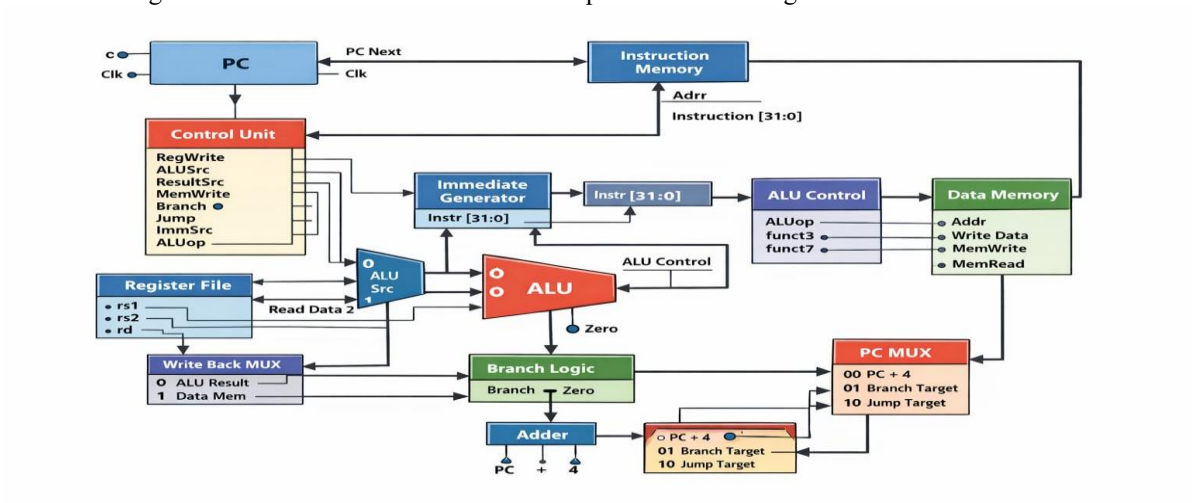
RISC-V is an open-source instruction set architecture widely used in modern processor design. A single-cycle processor executes each instruction in one clock cycle. In this project, a 32-bit RISC-V processor is implemented using Verilog HDL. The design includes datapath and control unit which together perform instruction fetch, decode, execute, memory access, and writeback. Simulation waveforms and synthesis schematics are used to verify correct operation of the processor.

3. Architecture / Block Diagram

The processor consists of the following main blocks:

• Program Counter
• Instruction Memory
• Control Unit
• Register File
• Immediate Generator
• ALU
• Data Memory
• Multiplexers
• PC Adder

The block diagram shows the connection between datapath and control signals.



Note:

This diagram represents the complete single-cycle RISC-V datapath. The current implementation covers core datapath components such as ALU, register file, memory, and control unit. Branch and jump PC update logic is not yet integrated and is planned as a future enhancement.

4. Modules Description

Program Counter – stores current instruction address
Instruction Memory – stores program instructions
Register File – stores registers
Immediate Generator – extracts immediate values
Control Unit – generates control signals
ALU – performs arithmetic and logic operations
ALU Control – selects ALU operation
Data Memory – read/write memory
Multiplexer – selects inputs
PC Adder – increments PC
Top Module – connects all modules

5. Verilog Design Code

The processor is implemented using multiple Verilog modules.
Each module is written separately and connected in the top module.

Modules included:
pc.v
instruction_memory.v
register_file.v
alu.v
imm_gen.v
control_unit.v
alu_control.v
data_memory.v
mux.v
riscv_top.v

```
`timescale 1ns / 1ps
// PC MODULE

module pc(input clk, input reset, input [31:0]pc_next, output reg [31:0]pc);
  always@(posedge clk or posedge reset)
    begin
      if(reset)
        pc <=0;
      else
        pc <=pc_next;
    end
endmodule

// PC ADDER

module pc_adder(input [31:0]pc ,output [31:0]pc_next);
```

```
  assign pc_next=pc+4;
endmodule

// INSTRUCTION MEMORY
module instruction_memory(

input [31:0] adder,

output [31:0] instruction
);

reg [31:0] mem [0:255];

integer i;

initial begin

// clear memory
for(i=0;i<256;i=i+1)
    mem[i] = 32'h00000013; // NOP

// program
mem[0] = 32'h00000033;
mem[1] = 32'h00100093;
mem[2] = 32'h00200113;
mem[3] = 32'h003081b3;

end

assign instruction = mem[adder[9:2]];

endmodue


module register_file(input clk, input we, input [4:0]rs1, input[4:0]rs2,
input[4:0]rd, input [31:0]wd , output[31:0] rd1, output[31:0] rd2);

  reg[31:0] regfile[0:31];
  integer i;
  initial begin
    for(i=0;i<32;i=i+1)
        regfile[i] = 0;
end

  assign rd1= regfile[rs1];
  assign rd2= regfile[rs2];

  always@ (posedge clk)
    begin
      if(we && rd !=0)
        regfile[rd]<=wd;
    end
endmodule


module alu(
```

```
    input [31:0] a,
    input [31:0] b,
    input [2:0] alu_ctrl,
    output reg [31:0] result,
    output zero
);

always @(*)
begin
    case(alu_ctrl)

        3'b000: result = a + b;
        3'b001: result = a - b;
        3'b010: result = a & b;
        3'b011: result = a | b;
        3'b100: result = a ^ b;

        default: result = 0;

    endcase
end

assign zero = (result == 0);

endmodule


module imm_gen(
    input [31:0] instr,
    input [1:0] imm_src,
    output reg [31:0] imm_out
);

always @(*)
begin
    case(imm_src)

        // I type
        2'b00:
            imm_out = {{20{instr[31]}}, instr[31:20]};

        // S type
        2'b01:
            imm_out = {{20{instr[31]}}, instr[31:25], instr[11:7]};

        // B type
        2'b10:
            imm_out = {{19{instr[31]}},
                       instr[31],
                       instr[7],
                       instr[30:25],
                       instr[11:8],
                       1'b0};

        // J type
```

```
        2'b11:
            imm_out = {{11{instr[31]}},
                       instr[31],
                       instr[19:12],
                       instr[20],
                       instr[30:21],
                       1'b0};

        default:
            imm_out = 0;

    endcase
end

endmodule


module control_unit(

input [6:0] opcode,

output reg RegWrite,
output reg MemWrite,
output reg MemRead,
output reg ALUSrc,
output reg ResultSrc,
output reg Branch,
output reg Jump,
output reg [1:0] ImmSrc,
output reg [1:0] ALUop

);

always @(*)
begin

    // default
    RegWrite = 0;
    MemWrite = 0;
    MemRead = 0;
    ALUSrc = 0;
    ResultSrc = 0;
    Branch = 0;
    Jump = 0;
    ImmSrc = 0;
    ALUop = 0;

    case(opcode)

    // R-type
    7'b0110011:
    begin
        RegWrite = 1;
        ALUSrc = 0;
        ResultSrc = 0;
```

```
        ALUop = 2'b10;
    end

    // I-type (addi)
    7'b0010011:
    begin
        RegWrite = 1;
        ALUSrc = 1;
        ImmSrc = 2'b00;
        ALUop = 2'b00;
    end

    // LW
    7'b0000011:
    begin
        RegWrite = 1;
        MemRead = 1;
        ALUSrc = 1;
        ResultSrc = 1;
        ImmSrc = 2'b00;
        ALUop = 2'b00;
    end

    // SW
    7'b0100011:
    begin
        MemWrite = 1;
        ALUSrc = 1;
        ImmSrc = 2'b01;
        ALUop = 2'b00;
    end

    // BEQ
    7'b1100011:
    begin
        Branch = 1;
        ImmSrc = 2'b10;
        ALUop = 2'b01;
    end

    // JAL
    7'b1101111:
    begin
        Jump = 1;
        RegWrite = 1;
        ImmSrc = 2'b11;
    end

    endcase

end

endmodule

module alu_control(
```

```
input [1:0] ALUop,
input [2:0] funct3,
input funct7,

output reg [2:0] alu_ctrl

);

always @(*)
begin

case(ALUop)

// for lw, sw, addi
2'b00:
    alu_ctrl = 3'b000;

// for beq
2'b01:
    alu_ctrl = 3'b001;

// R-type
2'b10:
begin

    case(funct3)

    3'b000:
        if(funct7)
            alu_ctrl = 3'b001;
        else
            alu_ctrl = 3'b000;

    3'b111:
        alu_ctrl = 3'b010;

    3'b110:
        alu_ctrl = 3'b011;

    3'b100:
        alu_ctrl = 3'b100;

    default:
        alu_ctrl = 3'b000;

    endcase

end

default:
    alu_ctrl = 3'b000;

endcase
```

```
end

endmodule


module data_memory(

input clk,
input MemWrite,
input MemRead,

input [31:0] addr,
input [31:0] wd,

output reg [31:0] rd

);

reg [31:0] mem [0:255];

always @(posedge clk)
begin
    if (MemWrite)
        mem[addr[9:2]] <= wd;
end

always @(*)
begin
    if (MemRead)
        rd = mem[addr[9:2]];
    else
        rd = 0;
end

endmodule

module riscv_top(

input clk,
input reset,

output [31:0] pc,
output [31:0] instruction,
output [31:0] alu_result,
output [31:0] mem_data

);

wire [31:0] pc_next;

wire [31:0] rd1;
wire [31:0] rd2;

wire [31:0] imm;
```

```
wire [2:0] alu_ctrl;

wire RegWrite;
wire MemWrite;
wire MemRead;
wire ALUSrc;
wire ResultSrc;
wire Branch;
wire Jump;

wire [1:0] ImmSrc;
wire [1:0] ALUop;

wire zero;



pc PC(
.clk(clk),
.reset(reset),
.pc_next(pc_next),
.pc(pc)
);


// PC + 4

assign pc_next = pc + 4;

instruction_memory IMEM(
.adder(pc),
.instruction(instruction)
);



control_unit CU(

.opcode(instruction[6:0]),

.RegWrite(RegWrite),
.MemWrite(MemWrite),
.MemRead(MemRead),
.ALUSrc(ALUSrc),
.ResultSrc(ResultSrc),
.Branch(Branch),
.Jump(Jump),
.ImmSrc(ImmSrc),
.ALUop(ALUop)

);



register_file RF(
```

```
.clk(clk),
.we(RegWrite),

.rs1(instruction[19:15]),
.rs2(instruction[24:20]),
.rd(instruction[11:7]),

.wd(ResultSrc ? mem_data : alu_result),

.rd1(rd1),
.rd2(rd2)

);



imm_gen IMM(

.instr(instruction),
.imm_src(ImmSrc),
.imm_out(imm)

);



alu_control ALUCTRL(

.ALUop(ALUop),
.funct3(instruction[14:12]),
.funct7(instruction[30]),

.alu_ctrl(alu_ctrl)

);



wire [31:0] alu_b;

assign alu_b = ALUSrc ? imm : rd2;



alu ALU(

.a(rd1),
.b(alu_b),
.alu_ctrl(alu_ctrl),

.result(alu_result),
.zero(zero)

);
```

```
data_memory DM(

.clk(clk),
.MemWrite(MemWrite),
.MemRead(MemRead),

.addr(alu_result),
.wd(rd2),
.rd(mem_data)

);

endmodule
```

6. Testbench

A SystemVerilog testbench is written to generate clock and reset. The testbench runs the processor for multiple cycles and displays PC, instruction, ALU result, and memory data.

```
`timescale 1ns / 1ps
module testbench;

logic clk;
logic reset;

logic [31:0] pc;
logic [31:0] instruction;
logic [31:0] alu_result;
logic [31:0] mem_data;


// DUT
riscv_top DUT(
.clk(clk),
.reset(reset),
.pc(pc),
.instruction(instruction),
.alu_result(alu_result),
.mem_data(mem_data)
);


// clock generation
always #5 clk = ~clk;


// reset and run
initial begin

clk = 0;
```

```
reset = 1;

#10;
reset = 0;

#200;

$finish;

end


// monitor
initial begin

$monitor(
"time=%0t clk=%b reset=%b pc=%0d instr=%h alu=%0d mem=%0d",
$time,
clk,
reset,
pc,
instruction,
alu_result,
mem_data
);

end

endmodule
```

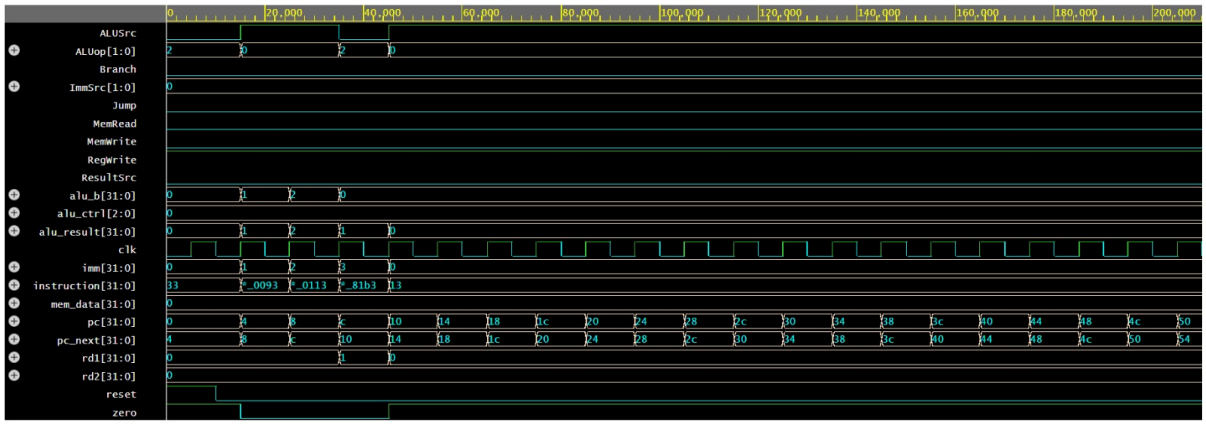
7. Simulation Result

The PC increments by 4 every clock cycle. Instructions are fetched from instruction memory. ALU performs correct operations. Control signals change according to instruction.

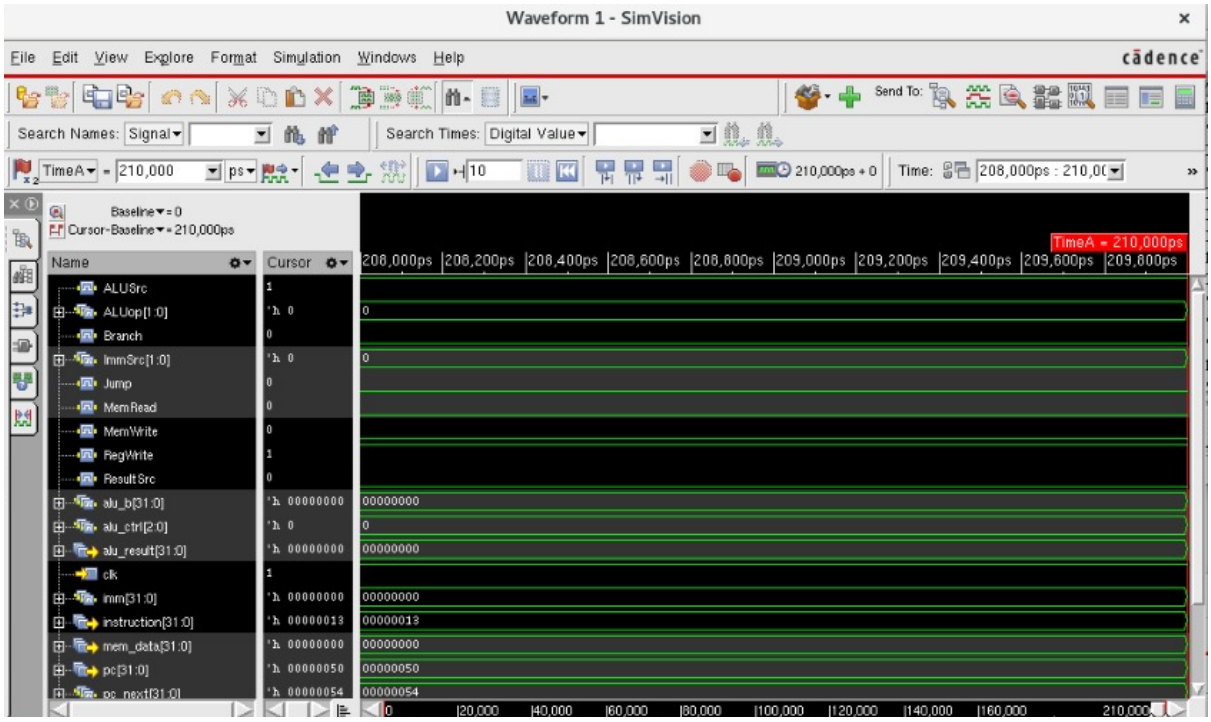
```
time=0 clk=0 reset=1 pc=0 instr=00000033 alu=0 mem=0
time=5000 clk=1 reset=1 pc=0 instr=00000033 alu=0 mem=0
time=10000 clk=0 reset=0 pc=0 instr=00000033 alu=0 mem=0
time=15000 clk=1 reset=0 pc=4 instr=00100093 alu=1 mem=0
time=20000 clk=0 reset=0 pc=4 instr=00100093 alu=1 mem=0
time=25000 clk=1 reset=0 pc=8 instr=00200113 alu=2 mem=0
time=30000 clk=0 reset=0 pc=8 instr=00200113 alu=2 mem=0
time=35000 clk=1 reset=0 pc=12 instr=003081b3 alu=1 mem=0
time=40000 clk=0 reset=0 pc=12 instr=003081b3 alu=1 mem=0
time=45000 clk=1 reset=0 pc=16 instr=00000013 alu=0 mem=0
time=50000 clk=0 reset=0 pc=16 instr=00000013 alu=0 mem=0
time=55000 clk=1 reset=0 pc=20 instr=00000013 alu=0 mem=0
time=60000 clk=0 reset=0 pc=20 instr=00000013 alu=0 mem=0
time=65000 clk=1 reset=0 pc=24 instr=00000013 alu=0 mem=0
time=70000 clk=0 reset=0 pc=24 instr=00000013 alu=0 mem=0
time=75000 clk=1 reset=0 pc=28 instr=00000013 alu=0 mem=0
time=80000 clk=0 reset=0 pc=28 instr=00000013 alu=0 mem=0
time=85000 clk=1 reset=0 pc=32 instr=00000013 alu=0 mem=0
time=90000 clk=0 reset=0 pc=32 instr=00000013 alu=0 mem=0
time=95000 clk=1 reset=0 pc=36 instr=00000013 alu=0 mem=0
time=100000 clk=0 reset=0 pc=36 instr=00000013 alu=0 mem=0
time=105000 clk=1 reset=0 pc=40 instr=00000013 alu=0 mem=0
time=110000 clk=0 reset=0 pc=40 instr=00000013 alu=0 mem=0
time=115000 clk=1 reset=0 pc=44 instr=00000013 alu=0 mem=0
time=120000 clk=0 reset=0 pc=44 instr=00000013 alu=0 mem=0
time=125000 clk=1 reset=0 pc=48 instr=00000013 alu=0 mem=0
time=130000 clk=0 reset=0 pc=48 instr=00000013 alu=0 mem=0
time=135000 clk=1 reset=0 pc=52 instr=00000013 alu=0 mem=0
time=140000 clk=0 reset=0 pc=52 instr=00000013 alu=0 mem=0
time=145000 clk=1 reset=0 pc=56 instr=00000013 alu=0 mem=0
time=150000 clk=0 reset=0 pc=56 instr=00000013 alu=0 mem=0
time=155000 clk=1 reset=0 pc=60 instr=00000013 alu=0 mem=0
time=160000 clk=0 reset=0 pc=60 instr=00000013 alu=0 mem=0
time=165000 clk=1 reset=0 pc=64 instr=00000013 alu=0 mem=0
time=170000 clk=0 reset=0 pc=64 instr=00000013 alu=0 mem=0
time=175000 clk=1 reset=0 pc=68 instr=00000013 alu=0 mem=0
time=180000 clk=0 reset=0 pc=68 instr=00000013 alu=0 mem=0
time=185000 clk=1 reset=0 pc=72 instr=00000013 alu=0 mem=0
time=190000 clk=0 reset=0 pc=72 instr=00000013 alu=0 mem=0
time=195000 clk=1 reset=0 pc=76 instr=00000013 alu=0 mem=0
time=200000 clk=0 reset=0 pc=76 instr=00000013 alu=0 mem=0
time=205000 clk=1 reset=0 pc=80 instr=00000013 alu=0 mem=0
testbench.sv:39: $finish called at 210000 (1ps)
```

9. Waveform

Two waveform results are shown.
The EPWave waveform shows full instruction execution.



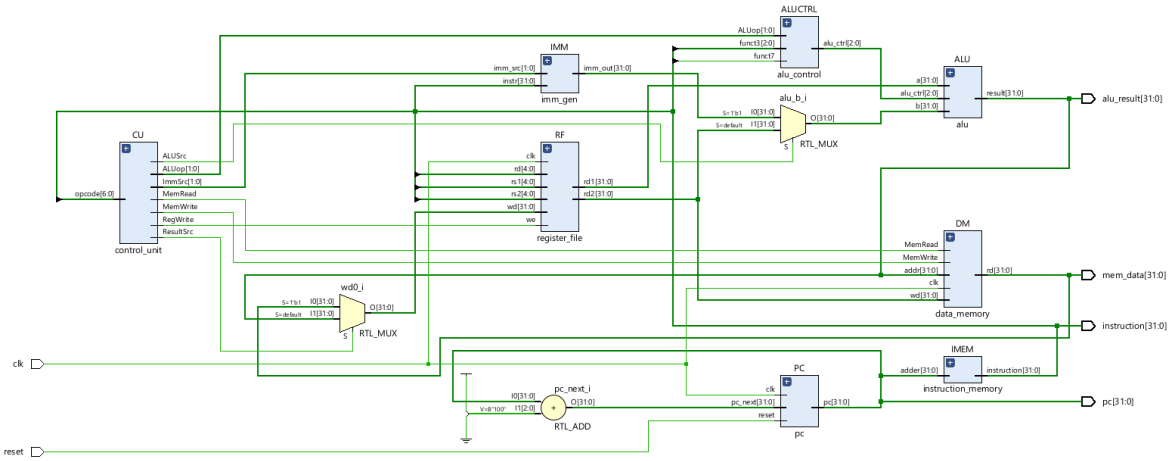
The Cadence waveform shows steady-state execution where the processor executes NOP instructions continuously. In this state, control signals remain constant, so the waveform appears as straight lines. This indicates stable processor operation.



10.RTL Schematic

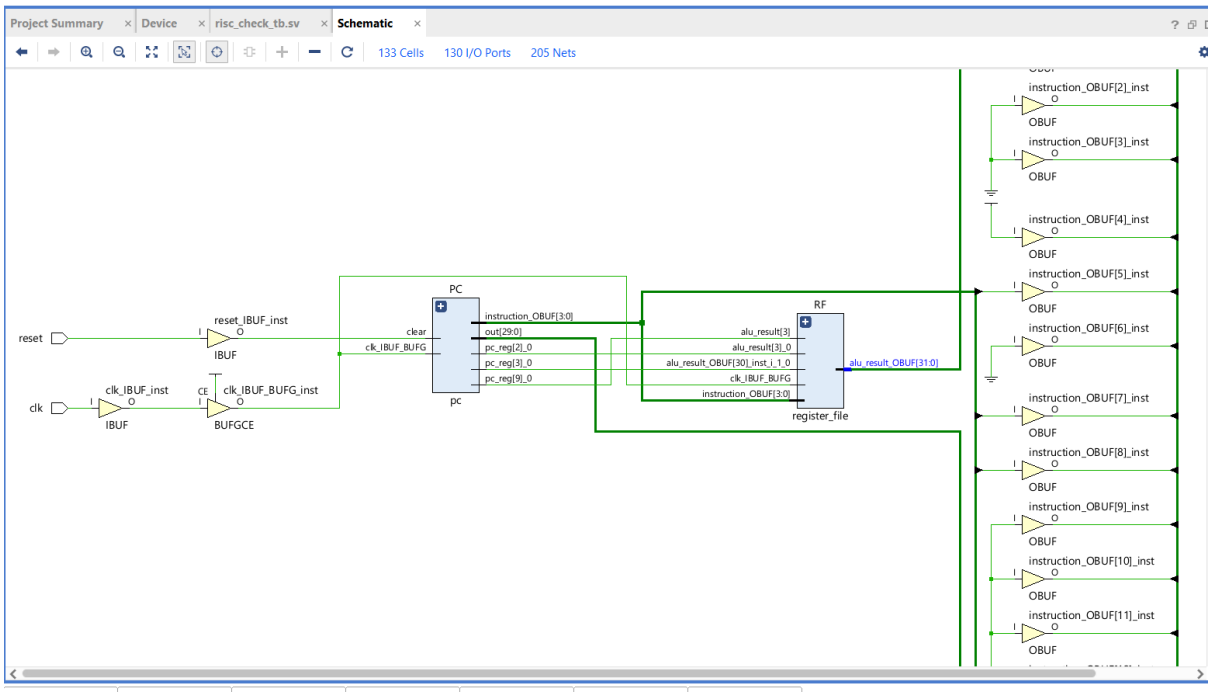
The RTL (Register Transfer Level) schematic shows the structural representation of the RISC-V single-cycle processor generated by the synthesis tool. This schematic represents how the Verilog HDL code is interpreted at the register-transfer level before gate-level synthesis. The RTL diagram confirms that all modules of the processor are correctly connected according to the datapath architecture. The RTL schematic verifies that the datapath and control path are correctly implemented and that the processor follows the RISC-V single-cycle

architecture. The successful generation of the RTL schematic confirms that the Verilog design is syntactically correct and structurally valid.



11. Synthesis Schematic

The schematic confirms correct conversion of the Verilog RTL code into hardware logic. Due to the large size of the processor design, only a partial view of the synthesized schematic is shown. The schematic includes important modules such as Program Counter, Register File, and datapath connections, which verifies that the processor is successfully synthesized.



12. Conclusion

The RISC-V single-cycle processor is successfully designed using Verilog. Simulation, waveform, RTL, and synthesis confirm correct operation. The project demonstrates understanding of processor architecture, datapath design, and hardware verification.